



Webfejlesztési keretrendszerek

7. gyakorlat

Dr. Jánki Zoltán Richárd

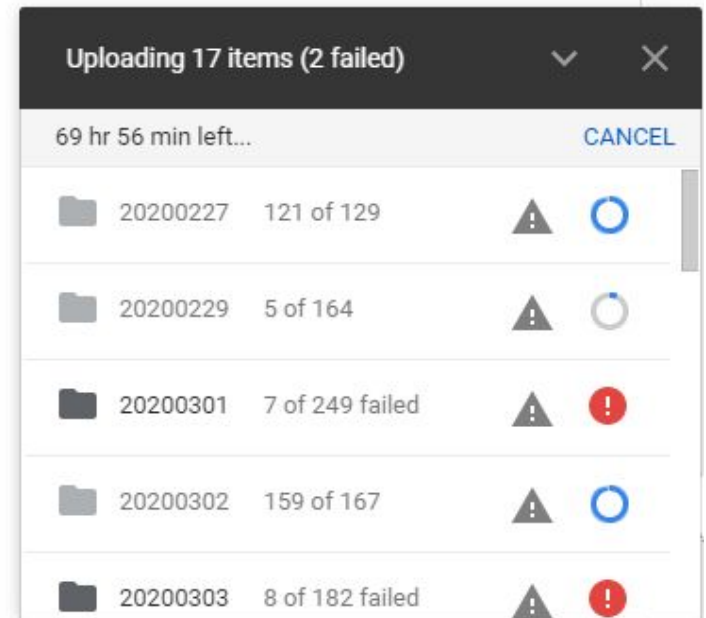
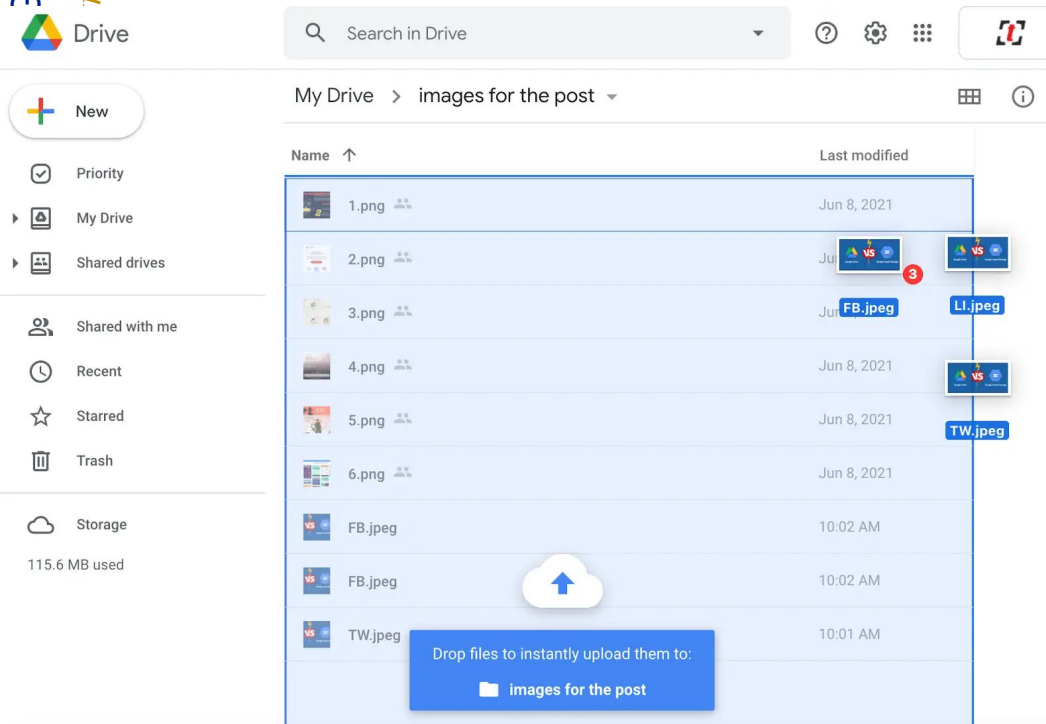
Hol tartunk?

- UI elkészült
- Routing elkészült
- Adatok
 - a kliens oldalon vannak in-memory jelleggel
 - lokális state kezelés
 - lazy-loading miatt az adat nem jut át más oldalakra



Hogyan jutnak el az adatok a klienshez?

- A logika jelentős része szinkron
- Az I/O-intenzív műveletek aszinkron módon történnek



Kérés-válasz interakció

- HTTP protokoll mentén
kérés-válasz
- Kérés
 - metódus, URL, fejléc, törzs
- Válasz
 - státuszkód, üzenet
- CRUD megfeleltetések
 - POST → Create
 - GET → Read
 - PUT/PATCH → Update
 - DELETE → Delete



2xx

Sikeres

200 OK
201 Created

A kérés teljesült.

3xx

Átírányítás

301 Moved
304 Not Modified

Máshol keressük.

4xx

Kliens hiba

400 Bad Request
404 Not Found

A kliens hibázott.

5xx

Szerver hiba

500 Internal Error
503 Unavailable

A szerver hibázott.

Aszinkron állapotok

- Loading (folyamatban)
 - spinner, skeleton loading
- Success (siker)
 - megjelenik a tartalom
 - üres tartalom nem hiba, de kezelendő
- Error (hiba)
 - retry gomb
 - specifikus felhasználó számára is értelmezhető üzenet (pl.: nem tiszta SQL hibakód)



Mindegyik állapotot kezelni kell UI oldalon!

Szinkron vs. aszinkron

```
const hetiEtrend = ["Gulyásleves", "Rántott hús"];

function etelHozzaadasa(ujEtel) {
    console.log("1. Étel hozzáadása a listához...");
    hetiEtrend.push(ujEtel);
    console.log("2. Étel elmentve a memóriába.");
}

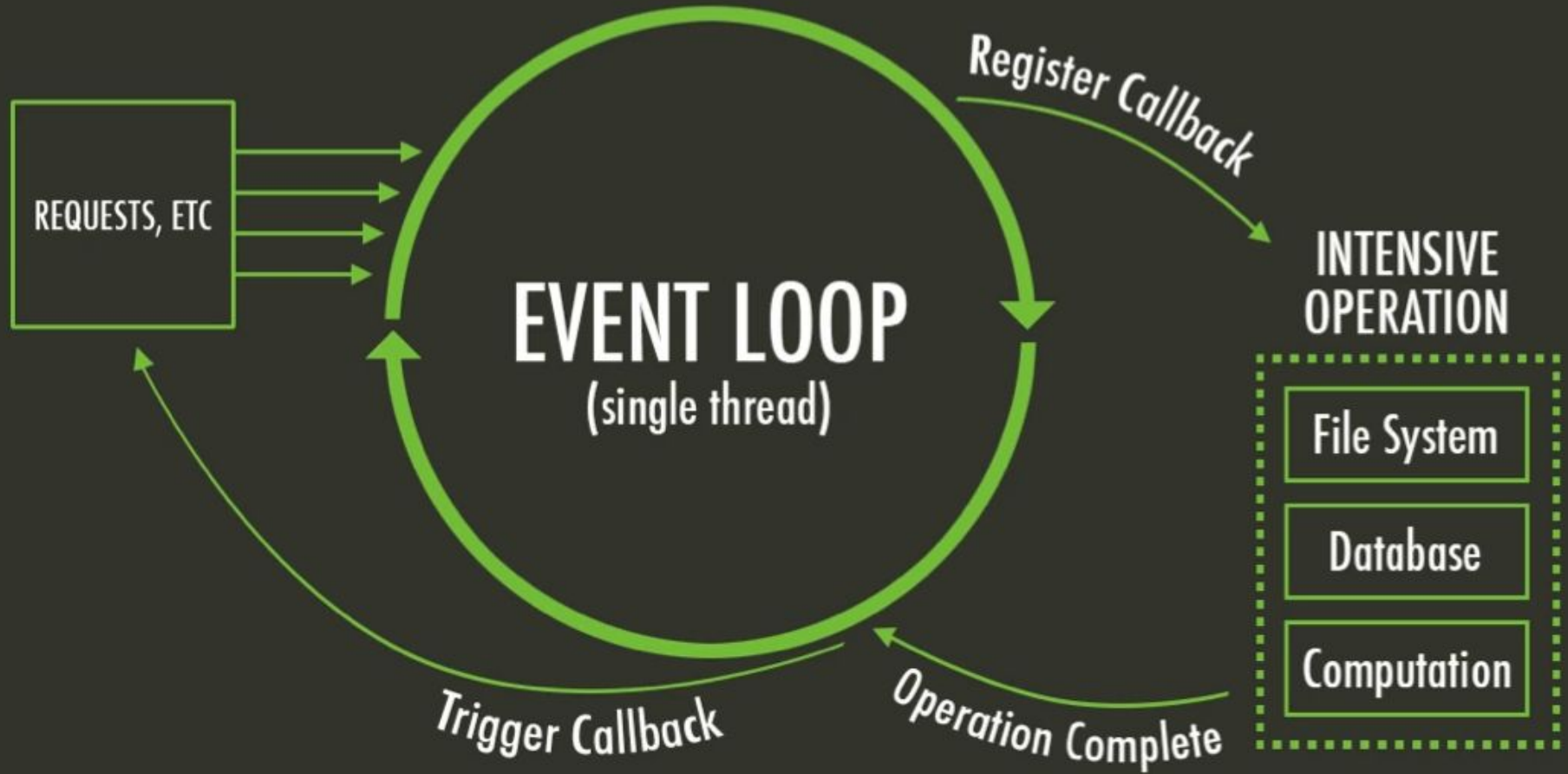
etelHozzaadasa("Túrós csusza");
console.log("3. Jelenlegi étrend:", hetiEtrend);
```

```
console.log("1. Étrendtervező indítása...");

// 0 ezredmásodperc = "Amint lehet, de csak ha végeztél a többivel"
setTimeout(() => {
    console.log("2. (Aszinkron) Receptek betöltése a gyorsítótárból...");
}, 0);

console.log("3. Mai menü megjelenítése: Rakott krumpli.");
```

NodeJS aszinkron modellje: EventLoop





Aszinkron műveletek kezelése

Callback

- Esemény bekövetkeztének elfogása
- Fejlesztői szabadság
 - mi történik, ha pozitív kimenetünk van?
 - mi történik, ha hiba van?

```
1 function hell(win) {  
2   // for listener purpose  
3   return function() {  
4     loadLink(win, REMOTE_SRC+'/assets/css/style.css', function() {  
5       loadLink(win, REMOTE_SRC+'/lib/async.js', function() {  
6         loadLink(win, REMOTE_SRC+'/lib/easyXDM.js', function() {  
7           loadLink(win, REMOTE_SRC+'/lib/json2.js', function() {  
8             loadLink(win, REMOTE_SRC+'/lib/underscore.min.js', function() {  
9               loadLink(win, REMOTE_SRC+'/lib/backbone.min.js', function() {  
10                loadLink(win, REMOTE_SRC+'/dev/base_dev.js', function() {  
11                  loadLink(win, REMOTE_SRC+'/assets/js/deps.js', function() {  
12                    loadLink(win, REMOTE_SRC+'/src/' + win.loader_path + '/loader.js', function() {  
13                      async.eachSeries(SRIPTS, function(src, callback) {  
14                        loadScript(win, BASE_URL+src, callback);  
15                      });  
16                    });  
17                  });  
18                });  
19              });  
20            });  
21          });  
22        });  
23      });  
24    });  
25  };  
26 }
```

Aszinkron műveletek kezelése

Callback példa

```
function mentes(nap, etel, callback) {  
  console.log(`Mentés folyamatban: ${etel} -> ${nap}...`);  
  setTimeout(() => {  
    // A callback függvényt hívjuk meg az eredménnyel  
    callback(`Siker: A ${etel} bekerült a ${nap}i menübe.`);  
  }, 1000);  
}  
  
mentes("Hétfő", "Lecsó", (uzenet) => {  
  console.log(uzenet);  
});
```

Aszinkron műveletek kezelése

Promise

- 3 állapot:
 - várakozik
 - teljesült
 - hibás
- Speciális függvények az esemény létrehozására
 - `resolve()` → pozitív kimenet
 - `reject()` → hiba
- Speciális függvények az esemény kezelésére
 - `then()` → pozitív ág
 - `catch()` → hiba
 - `finally()` → függetlenül a kimenettől még valami fusson le



DE! Még mindig aszinkron kódunk van! :(

Aszinkron műveletek kezelése

Promise példa

```
function mentesPromise(nap, etel) {  
  return new Promise((resolve, reject) => {  
    console.log(`Mentés folyamatban: ${etel}...`);  
    setTimeout(() => {  
      if (!etel) {  
        reject("Hiba: Üres ételnevet nem menthetünk!");  
      } else {  
        resolve(`Siker: A ${etel} elmentve ${nap}ra.`);  
      }  
    }, 1000);  
  });  
}  
  
mentesPromise("Hétfő", "Lecsó")  
  .then(uzenet => console.log(uzenet))  
  .catch(hiba => console.error(hiba));
```

Aszinkron műveletek kezelése

Async-await

- Aszinkron kódból szinkron kód készítése
- Függvény fejlécben megjelöljük, hogy aszinkron kódot tartalmaz a blokk (async kulcsszó)
- Az aszinkron hívás végét bevárjuk (await kulcsszó)
- Hibakezelés try/catch blokkokkal történik
- Az async-nak megjelölt függvények is Promise-szal térnek vissza



Aszinkron műveletek kezelése

Async-await példa

```
async function etrendFrissitese() {  
  try {  
    const uzenet = await mentesPromise("Hétfő", "Lecsó");  
    console.log(uzenet);  
  } catch (hiba) {  
    console.error("Valami elromlott: " + hiba);  
  }  
}  
  
etrendFrissitese();
```

Aszinkron műveletek kezelése

Observable

- Reaktív programozás alapja (RxJS)
- Adatfolyamok használata
 - Nyitunk egy adatfolyamot
 - Küldünk benne valamennyi adatot
 - Lezárjuk az adatfolyamot
- Az adatfolyamra feliratkozhatunk (subscribe), így jutunk hozzá az adatokhoz
 - next → adat érkezésekor futunk ide
 - error → hiba esetén futunk ide (a stream lezárul)
 - complete → a folyam utolsó eleme utáni blokk
- Feliratkozásról gondoskodni kell (unsubscribe)
 - potenciális memory-leak hiba!



Ezen az elven működik a HTTP protokoll implementációja is!

Aszinkron műveletek kezelése

Observable

```
const mentesObservable = (nap, etel) => new Observable(subscriber => {
  console.log(`Mentés folyamatban: ${etel}...`);
  setTimeout(() => {
    if (!etel) {
      subscriber.error("Hiba: Nincs megadva recept!");
    } else {
      subscriber.next(`Siker: A ${etel} elmentve ${nap}ra.`);
      subscriber.complete();
    }
  }, 1000);
});

mentesObservable("Hétfő", "Lecsó").subscribe({
  next: uzenet => console.log(uzenet),
  error: hiba => console.error(hiba)
});
```

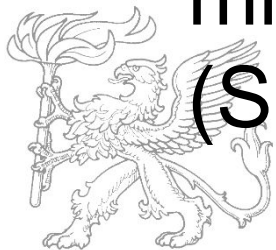
Versenyhelyzet (race condition)

- Ha két aszinkron művelet között versenyhelyzet van, az eredmény nem determinisztikus
- Javaslatok
 - aszinkron műveletek közötti függőségek tisztázása (egymás után indulnak)
 - AbortController: új kérés indítása előtt az előzőt leállítjuk



Helyi tárolás

- Gyakran vagy több helyen használandó adatokat érdemes helyben is eltárolni
 - localStorage (érzékeny adatot ne itt!)
 - sessionStorage
 - IndexedDB
- Ahol kritikus a friss adat megléte, ott mindig forduljunk az “igazság forrásához” (SSOT)



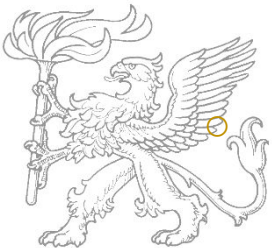
Cache stratégiák

- Cache-first
 - először a gyorsítótárból szolgáljuk ki a kérést
- Network-first
 - először szervertől kérünk adatot
- Network-only
 - nincs gyorsítótárazás a kliensnél



Google Cloud Firebase

- Google felhő-alapú platform
- Felhő-alapú szolgáltatások
 - Felhasználókezelés
 - Firebase Authentication
 - Adatbázis
 - Firebase Cloud Firestore (NoSQL)
 - Tároló
 - Firebase Cloud Storage (binárisokhoz)
 - Hosting
 - Firebase Hosting (HTTPS-alapú web-app host-olás)



Adatbázis és adatmodell

- Sémamentes, JSON-alapú struktúra
- Kollektciók, dokumentumok, mezők
- GUI-orientált adatmodell tervezés
 - amit látunk egy képernyőn, azt érdemes egy dokumentumba tenni (!), így spórolhatunk a lekérdezések számán
 - a számításokat ne a kliens végezze, hanem az adatbázis
 - pl.: hány eleme van a kollekciónak (van count művelet)



Gyakorlat

- Hozzuk létre az adatbázisunkat!
 - Firebase projekt és Firestore adatbázis
 - Tervezzük meg az adatmodellt (a GUI mentén)
 - Töltsük fel adatokkal az adatbázist
- Firestore bekötése a kliens alkalmazásba
 - Környezeti változók beállítása
 - Service-ekből indítsuk el a lekérdezéseket

